



UNIVERSITATEA DIN BUCUREȘTI

**FACULTATEA DE
MATEMATICĂ ȘI
INFORMATICĂ**

SPECIALIZAREA INFORMATICĂ



Lucrare de licență

Gestionarea securizată a datelor medicale folosind Blockchain și Criptografie

Absolvent

Veisa Radu-George

Coordonator științific

Conf. univ. Dr. Marius-Iulian Mihăilescu

București, iunie 2026

Rezumat

Scopul principal al lucrării a fost dezvoltarea unui sistem software pentru stocarea securizată a documentelor medicale, accentul căzând pe protejarea confidențialității datelor și pe acordarea unei metode de control real asupra informațiilor medicale pentru utilizatori. Arhitectura este proiectată astfel încât server-ul să nu aibă acces la date în clar, criptarea fiind realizată exclusiv în browser-ul utilizatorului.

Au fost implementate roluri distincte pentru pacienți, medici și administratori, astfel încât accesul la informație să fie realizat doar pe baza permisiunilor definite și a consimțământului pacienților, iar integritatea documentelor și trasabilitatea accesului au fost asigurate prin înregistrarea informațiilor relevante pe blockchain.

Abstract

The main objective of this thesis was the development of a software system for the secure storage of medical records, with a focus on protecting data confidentiality and providing users with real control over their own medical information. The architecture is designed so that the server never has access to data in plain text, as encryption is performed exclusively in the user's browser.

Distinct roles were implemented for patients, doctors and administrators, ensuring that access to information is granted solely based on the defined permissions and patient consent, while document integrity and access traceability are ensured by recording the relevant information on the blockchain.

Cuprins

Capitolul 1 - Introducere	5
1.1 Motivație.....	5
1.2 Structura lucrării	5
Capitolul 2 - Preliminarii	6
2.1 Tehnologii blockchain.....	7
2.2 Criptografie aplicată	7
Capitolul 3 - Descrierea soluției	10
3.1 Prezentarea sistemului	10
3.2 Modelul de date	11
3.3 Implementarea modulelor.....	14
3.3.1 Managementul cheilor și autentificarea	14
3.3.2 Încărcarea și stocarea documentelor	15
3.3.3 Descărcarea și verificarea integrității.....	17
3.3.4 Consimțământul și auditul accesului.....	18
3.3.5 Verificarea medicilor și panoul de admin	19
Capitolul 4 - Analiza securității și a fezabilității.....	22
4.1 Analiza amenințărilor	22
4.2 Confidențialitatea datelor și conformitatea GDPR.....	23
4.3 Analiza costurilor de tranzacție	23
Capitolul 5 - Concluzii	25
Bibliografie	26

Capitolul 1 - Introducere

1.1 Motivație

Stocarea și gestionarea datelor sunt unele din cele mai mari provocări cu care se confruntă sistemele informatice actuale, mai ales în sectorul medical, unde confidențialitatea este critică. Datele medicale includ informații despre diagnostice, tratamente și alte aspecte personale ale pacienților, a căror expunere și modificare neautorizată poate avea consecințe grave asupra vieții private.

În prezent, accesul pacienților la propriile date medicale este realizat, în majoritatea cazurilor, prin intermediul unor platforme de tip portal oferite de companii specializate precum Epic Systems Corporation, dezvoltatorii MyChart (Epic Systems Corporation, 2026) sau Oracle Corporation, dezvoltatorii Oracle Health Patient Portal (Oracle Health, n.d.). Aceste soluții permit vizualizarea informațiilor și a documentelor clinice, însă funcționează în cadrul unor arhitecturi centralizate, în care datele sunt stocate și administrate pe infrastructura furnizorului sau a instituției medicale partenere, lucru ce presupune că protecția confidențialității depinde în mod direct de politicile și mecanismele de securitate ale operatorului platformei.

Această lucrare propune ca soluție dezvoltarea unei platforme web pentru gestionarea securizată a documentelor medicale care oferă utilizatorilor control asupra acestora și care reduce dependența de intermediari. Sistemul utilizează criptare la nivel de browser și acordă server-ului rolul de coordonare și gestionare a metadatelor, iar fișierele sunt stocate într-un sistem distribuit de tip IPFS. Pentru implementarea mecanismului de gestionare a cheilor criptografice am ales să utilizez o strategie de derivare a cheii private din parola utilizatorului folosind PBKDF2 pentru a o face inaccesibilă server-ului în formă necriptată și dependentă de autentificarea utilizatorului. Astfel, chiar dacă un atacator obține acces la baza de date, cheia privată nu poate fi utilizată fără cunoașterea parolei utilizatorului.

1.2 Structura lucrării

Lucrarea cuprinde capitole care acoperă atât tehnologiile și conceptele utilizate, cât și implementarea și testarea sistemului:

Preliminarii, cu următoarele subcapitole:

- **Tehnologii blockchain**, în care sunt prezentate conceptele de blockchain și smart contracts și integrarea acestora în lucrare;
- **Criptografie aplicată**, în care sunt descriși algoritmi utilizați (AES, RSA, PBKDF2, SHA-256) precum și schema hibridă de criptare aleasă pentru implementarea sistemului;

Descrierea soluției, cu următoarele subcapitole:

- **Prezentarea sistemului**, în care este descrisă arhitectura și tehnologiile folosite;
- **Modelul de date**, în care este descrisă baza de date și smart contracts;
- **Implementarea modulelor**, în care sunt prezentate funcționalitățile principale ale platformei;

Analiza securității, cu următoarele subcapitole:

- **Analiza amenințărilor**, în care sunt identificate amenințările relevante prin metodologia STRIDE și sunt descrise contramăsurile implementate;
- **Confidențialitatea datelor**, în care este abordat felul în care au fost transpuse dreptul la ștergere și dreptul de retragere a consimțământului în platformă;
- **Analiza costurilor de tranzacție**, în care sunt explorate costurile de gas și fezabilitatea economică a platformei;

Concluzii, alături de limitările cunoscute ale soluției și direcțiile de dezvoltare ulterioară.

Capitolul 2 - Preliminarii

2.1 Tehnologii blockchain

Conceptul de blockchain - registru distribuit bazat pe lanțuri de blocuri - a fost introdus ca soluție a problemei de cheltuială dublă în tranzacțiile electronice fără necesitatea unei autorități centrale de validare. Fiecare bloc conține hash-ul blocului anterior, formând un lanț în care modificarea unui bloc necesită refacerea blocurilor ulterioare (Nakamoto, 2008).

Ethereum extinde acest concept prin introducerea de smart contracts. Un smart contract este, în esență, un program care rulează pe blockchain-ul Ethereum. Acestea funcționează ca un tip special de cont, capabil să dețină un sold și să participe la tranzacții în cadrul rețelei. Spre deosebire de conturile controlate de utilizatori, acestea funcționează autonom pe baza codului prin care au fost implementate (Ethereum Foundation, 2026).

Una dintre limitările blockchain-ului este că acesta nu este conceput pentru stocarea fișierelor de dimensiuni mari. Pentru a evita această problemă, am utilizat smart contracts pentru a înregistra referințele fișierelor pe blockchain și am utilizat un sistem extern de stocare pentru conținutul propriu-zis al acestora. Am ales să folosesc un sistem IPFS (InterPlanetary File System) deoarece este foarte compatibil cu tehnologia blockchain prin faptul că utilizează hash-uri generate pe baza conținutului fișierelor numite CID-uri (Content Identifier) care se modifică atunci când conținutul este alterat (Protocol Labs, n.d.).

2.2 Criptografie aplicată

Algoritmii criptografici sunt utilizați în majoritatea sistemelor informatice moderne pentru a garanta confidențialitatea și integritatea datelor. Printre acești algoritmi se numără și cei utilizați în implementarea platformei: AES, SHA-256, PBKDF2 și RSA-OAEP.

AES (Advanced Encryption Standard) este un algoritm de criptare care operează pe

blocuri de 128 de biți și suportă chei de 128, 192 și 256 de biți. Algoritmul transformă datele printr-o succesiune de runde, fiecare conținând operații de substituție, permutare și combinare cu cheia de rundă (National Institute of Standards and Technology, 2001). Varianta AES-256, utilizată de sistemul din această lucrare, aplică 14 runde de transformări, oferind securitate pe termen lung împotriva atacurilor prin spațiul mare de chei.

SHA-256 este o funcție hash din familia funcțiilor SHA-2 care primește un mesaj de lungime arbitrară și produce un rezumat criptografic de o lungime fixă de 256 de biți. Aceasta garantează că din hash-ul rezultat nu se poate reconstitui mesajul original și că nu se pot găsi două mesaje diferite cu același hash (National Institute of Standards and Technology, 2015).

PBKDF2 (Password-Based Key Derivation Function 2) este o funcție de derivare a cheilor criptografice din parole. Algoritmul combină parola cu un salt aleator și aplică în mod repetat o funcție pseudoaleatorie, obținând cheia de lungime dorită (Moriarty, Kaliski, & Rusch, 2017). Cu cât numărul de iterații este mai mare, cu atât mai mult crește costul computațional al derivării, făcând atacurile de tip brute-force mult mai dificile.

RSA-OAEP este un algoritm de criptare care utilizează o pereche de chei: o cheie publică, distribuită liber, și una privată, păstrată secret. Cheile publice sunt folosite la criptarea mesajelor pentru a face posibilă decriptarea doar prin intermediul cheii private corespunzătoare (Jonsson & Kaliski, 2003).

În proiectarea sistemului am ales să utilizez o schemă de criptare hibridă deoarece aceasta combină eficiența criptografiei simetrice cu securitatea distribuției de chei oferită de criptografia asimetrică. Limitarea RSA-OAEP provine din construcția algoritmului: pentru un modul de 2048 de biți ($k = 256$ octeți) și o funcție hash SHA-256 ($hLen = 32$ octeți), dimensiunea maximă a mesajului ce poate fi criptat într-o singură operație este $k - 2 \times hLen - 2 = 190$ de octeți (Jonsson & Kaliski, 2003). Documentele medicale utilizate în platformă au dimensiuni de ordinul sutelor de KB până la câțiva MB, depășind cu mult această limită. În schimb, o cheie AES-256 are doar 32 de octeți, încadrându-se perfect în capacitatea RSA-OAEP.

Pe baza acestei observații, schema de criptare hibridă aleasă calculează pentru fiecare document M amprenta de integritate $H = \text{SHA} - 256(M)$ și generează aleatoriu o cheie simetrică K de 256 de biți și un vector de inițializare IV de 96 de biți. Ulterior, conținutul documentului este criptat simetric folosind AES-256 în modul GCM: $(C, T) = \text{AES} - 256 - \text{GCM_Enc}(K, IV, M)$, unde C este textul cifrat, iar T este tag-ul de autentificare de 128 de biți generat automat, garantând atât confidențialitatea, cât și detectarea oricărei modificări. Cheia simetrică K este apoi criptată asimetric cu cheia publică RSA-OAEP a pacientului: $K_{\text{enc}} = \text{RSA} - \text{OAEP_Enc}(PK_{\text{pacient}}, K)$, iar informațiile criptate $(C, T, IV, K_{\text{enc}}, H)$ sunt transmise către server, acesta neavând acces în niciun moment la M sau K . La accesarea documentului, procesul se desfășoară invers: cheia privată RSA este decriptată local în browser și se obține $K = \text{RSA} - \text{OAEP_Dec}(SK_{\text{pacient}}, K_{\text{enc}})$, unde SK_{pacient} este cheia privată a pacientului, și apoi $M = \text{AES} - 256 - \text{GCM_Dec}(K, IV, C, T)$. GCM verifică automat tag-ul T la decriptare și respinge operația dacă textul cifrat a fost alterat.

Capitolul 3 - Descrierea soluției

3.1 Prezentarea sistemului

Arhitectura este alcătuită din cinci componente principale: backend, care coordonează comunicarea între componente fără a accesa conținutul decriptat al documentelor, frontend, unde se realizează operațiile criptografice, baza de date, care stochează informațiile utilizatorilor, metadatele documentelor, starea consimțământului și auditul off-chain, sistemul de stocare IPFS, unde sunt stocate documentele criptate și rețeaua blockchain locală, pe care sunt înregistrate referințele criptografice și starea consimțământului.

Server-ul a fost creat utilizând Node.js și framework-ul Express.js, alături de următoarele pachete:

- JSON Web Tokens (JWT) și cookie-parser, folosite pentru a asigura autentificarea și gestionarea sesiunilor prin cookie-uri de tip httpOnly și pentru a preveni atacuri;
- bcrypt, utilizat pentru securizarea parolelor utilizatorilor;
- Axios și form-data, folosite pentru a gestiona comunicarea HTTP cu serviciile externe și pentru sistemul de încărcare a documentelor.

Interfața pentru utilizatori a fost dezvoltată folosind React împreună cu Vite, integrând următoarele tehnologii:

- React Router DOM, utilizat pentru gestionarea navigării interne de tip Single Page Application;
- TailwindCSS, folosit cu scopul de a realiza stilizări pentru interfață;
- Web Crypto API, utilizat pentru executarea operațiilor de criptare și decriptare direct în browser-ul utilizatorului.

Pentru stocarea datelor off-chain am ales să folosesc o bază relațională de date PostgreSQL, care este rulată într-un container izolat de Docker, și Prisma ORM pentru definirea schemei bazei de date și generarea automată a tabelor.

Logica de validare și înregistrare a datelor a fost implementată folosind:

- Solidity, pentru scrierea de smart contracts;
- Hardhat, la realizarea mediului de dezvoltare pentru blockchain-ul local;
- ethers.js, pentru comunicarea dintre backend și blockchain.

Fișierele criptate sunt stocate în afara server-ului folosind Pinata API pentru asigurarea persistenței și a stocării descentralizate a documentelor.

3.2 Modelul de date

Stocarea datelor se realizează pe două niveluri: nivelul off-chain, reprezentat de baza de date PostgreSQL și de sistemul de stocare cu Pinata, și nivelul on-chain, bazat pe smart contracts.

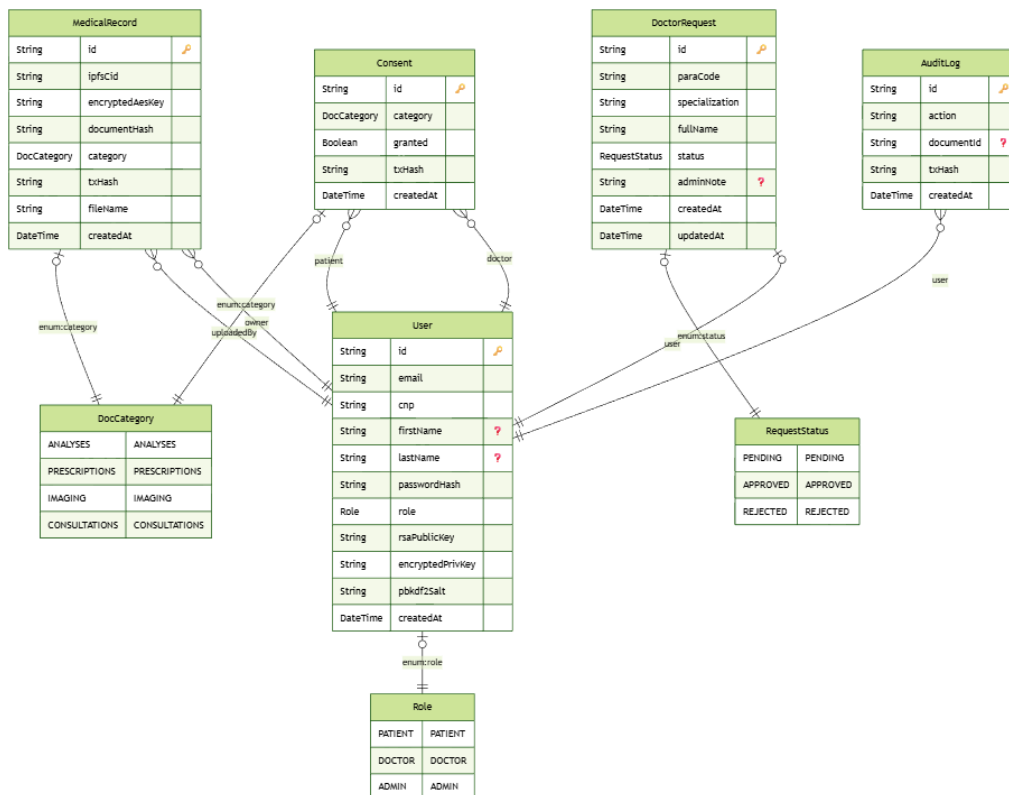


Figura 3.2.1: Schema bazei de date

Baza de date relațională este structurată în jurul a cinci entități principale. Nucleul este reprezentat de tabela User deoarece aceasta stochează datele de identificare și informațiile criptografice ale utilizatorilor și salvează și nivelul de acces în platformă,

folosind enum-ul Role. Detaliile documentelor sunt reținute în MedicalRecord, tabela care păstrează adresa IPFS, cheia AES criptată și hash-urile pentru document și pentru tranzacția pe blockchain. Clasificarea documentelor se realizează cu ajutorul enumerării DocCategory. Tabela Consent are rolul de a gestiona permisiunile de acces ale utilizatorilor la nivel de categorie și de a stoca starea consimțământului. Procesul de validare a identității unui medic este susținut de entitatea DoctorRequest, care stochează informațiile din cererile trimise către administratori și urmărește starea acestora prin enum-ul RequestStatus. AuditLog este utilizat pentru a păstra informațiile cu privire la resursele accesate de utilizatori, garantând trasabilitatea operațiunilor.

În ceea ce privește componenta on-chain, am implementat trei contracte în Solidity pentru a înregistra referințele datelor și pentru a garanta un istoric de acces care să nu poată fi modificat sau șters.

Contractul MedicalRecords a fost implementat pentru a salva direct pe blockchain CID-ul IPFS, hash-ul SHA-256 al documentului, ID-ul înregistrării și adresa contului de blockchain folosită de sistem. Sistemul a fost proiectat plecând de la premisa că pacienții nu dețin portofele individuale și nu interacționează cu blockchain-ul în mod direct. Această decizie a fost luată pentru a simplifica procesul de înregistrare, utilizatorii nemaifiind nevoiți să dețină ETH sau să gestioneze chei private de portofel. Structura de date utilizată pentru a încapsula aceste atribute pe blockchain este prezentată în figura 3.2.2.

```
4  contract MedicalRecords {
5      struct Record {
6          string ipfsCid;
7          bytes32 documentHash;
8          address owner;
9          address uploadedBy;
10         uint8 category;
11         uint256 timestamp;
12     }
```

Figura 3.2.2: Structura de date pentru înregistrarea documentelor medicale

Contractul Consent reprezintă baza sistemului de permisiuni. Acesta mapează pacienții,

medicii și categoriile pentru a stoca starea consimțământului. Modul în care este gestionată această stare la nivelul contractului, folosind o structură de date de tip dicționar imbricat (mapping), este ilustrat în figura 3.2.3. Astfel, starea poate fi interogată direct on-chain, prin funcția `hasConsent`, pentru a verifica dreptul medicilor de a accesa o anumită categorie de documente a unui pacient.

```
4 contract Consent {
5     mapping(string => mapping(string => mapping(uint8 => bool))) private consents;
6
7     event ConsentGranted(string indexed patient, string indexed doctor, uint8 category);
8     event ConsentRevoked(string indexed patient, string indexed doctor, uint8 category);
9 }
```

Figura 3.2.3: Definirea structurii de mapping pentru gestionarea permisiunilor

Contractul `AuditLog` asigură trasabilitatea sistemului și transparența totală a operațiunilor prin caracterul imuabil specific blockchain, având rol de registru. Orice încercare de vizualizare a documentelor va declanșa funcția de logare care generează un eveniment on-chain. Definirea acestui eveniment, detaliată în figura 3.2.4, demonstrează cum sunt salvate datele esențiale ale tranzacției: identificatorul utilizatorului, ID-ul documentului, tipul acțiunii și marcajul de timp (timestamp) generat automat pe rețea.

```
4 contract AuditLog {
5     event AccessLogged(
6         string indexed accessor,
7         string indexed recordId,
8         string action,
9         uint256 timestamp
10    );
11 }
```

Figura 3.2.4: Evenimentul utilizat pentru auditul operațiunilor

Pentru a garanta siguranța logicii on-chain, contractele au fost testate folosind bibliotecile `Chai` și `ethers.js`. Scenariile de testare validează emiterea corectă a evenimentelor pe platformă și modificarea stării, garantând independența categoriilor de consimțământ și asigurând respingerea operațiunilor nepermise.

3.3 Implementarea modulelor

3.3.1 Managementul cheilor și autentificarea

Sistemul de gestionare a cheilor criptografice este o componentă foarte importantă a arhitecturii deoarece de acesta depinde întreaga garanție de confidențialitate a documentelor medicale. Toate operațiile de criptare și decriptare sunt realizate în browser prin Web Crypto API, server-ul primind și stocând doar datele deja criptate.

Acest principiu este surprins încă din etapa de creare a contului unui utilizator. Înregistrarea unui cont nou se realizează printr-un formular care solicită adresa de email, CNP-ul și parola utilizatorului (vezi figura 3.3.1.1). Înainte de a se trimite aceste date către server se generează o pereche de chei RSA-OAEP pe 2048 de biți și un exponent public cu valoarea 65537, conform standardelor propuse de NIST (Barker & Roginsky, 2019). Valoarea exponentului garantează o viteză de criptare mult mai mare și este suficient de mare pentru a preveni atacuri matematice.

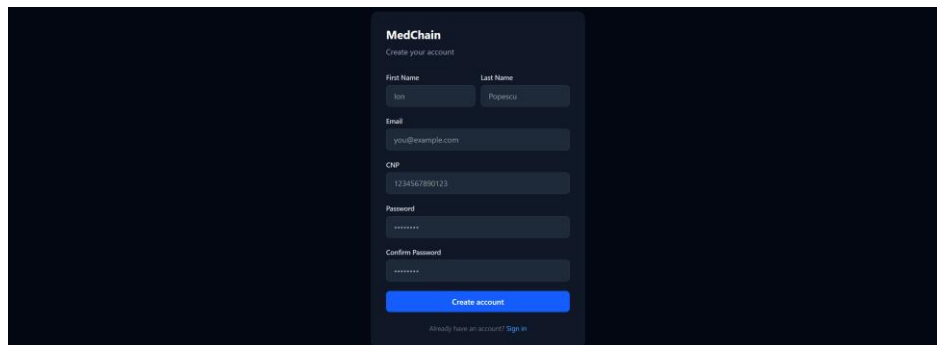
The image shows a registration form for 'MedChain' with the title 'Create your account'. The form is set against a dark background. It includes input fields for 'First Name' (containing 'Ion'), 'Last Name' (containing 'Popescu'), 'Email' (containing 'you@example.com'), 'CNP' (containing '1234567890123'), 'Password' (masked with dots), and 'Confirm Password' (also masked). A blue 'Create account' button is at the bottom, with a link 'Already have an account? Sign in' below it.

Figura 3.3.1.1: Portalul de register

Pentru a securiza cheia privată RSA înainte de a o trimite către server, am generat un salt aleatoriu de 16 octeți și am aplicat funcția PBKDF2 cu 200.000 de iterații și cu HMAC-SHA256, o funcție pseudoaleatorie, asupra parolei utilizatorului generând astfel o cheie simetrică AES-256. Aceasta este utilizată pentru a cripta cheia RSA alături de un vector de inițializare de 96 de biți. După finalizarea acestor operații, datele utilizatorului sunt trimise către server alături de noile date criptografice.

Procesul de autentificare implementează fluxul invers. Utilizatorul introduce datele de

conectare, iar server-ul le validează folosind funcția `bcrypt`. Dacă validarea este reușită, backend-ul returnează saltul și cheia privată și browser-ul regenerează cheia AES. Browser-ul decriptează apoi cheia privată RSA și o încarcă în memoria volatilă a aplicației, ca variabilă de stare în React. Managementul sesiunii este asigurat printr-un token JWT, setat ca un cookie `httpOnly` cu atributul `SameSite=Strict` pentru protecția CSRF.

Această alegere de implementare presupune că la închiderea tab-ului sau la reîncărcarea paginii, cheia privată este distrusă automat. Pentru a asigura o experiență de utilizare fluidă, am implementat un mecanism de re-autentificare rapidă care verifică validitatea token-ului JWT și prezintă utilizatorului un modal (vezi figura 3.3.1.2) care solicită doar parola pentru re-derivare.

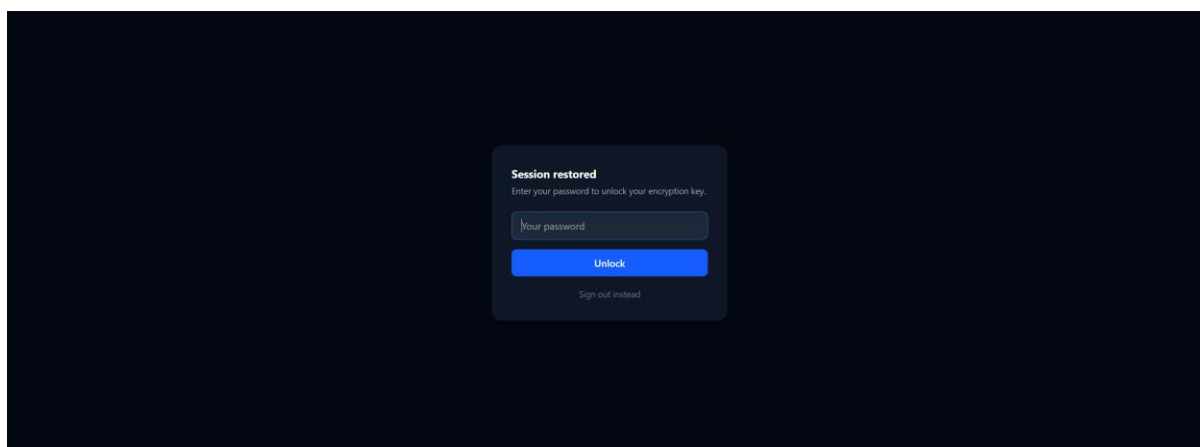


Figura 3.3.1.2: *Modalul de re-autentificare*

3.3.2 Încărcarea și stocarea documentelor

Așa cum este ilustrat în diagrama de secvență din figura 3.3.2.1, procesul de încărcare a unui document medical presupune colaborarea dintre componentele responsabile de operațiile criptografice din browser, server-ul backend, sistemul de stocare IPFS și contractul inteligent `MedicalRecords`. Etapele acestui proces sunt detaliate în continuare.

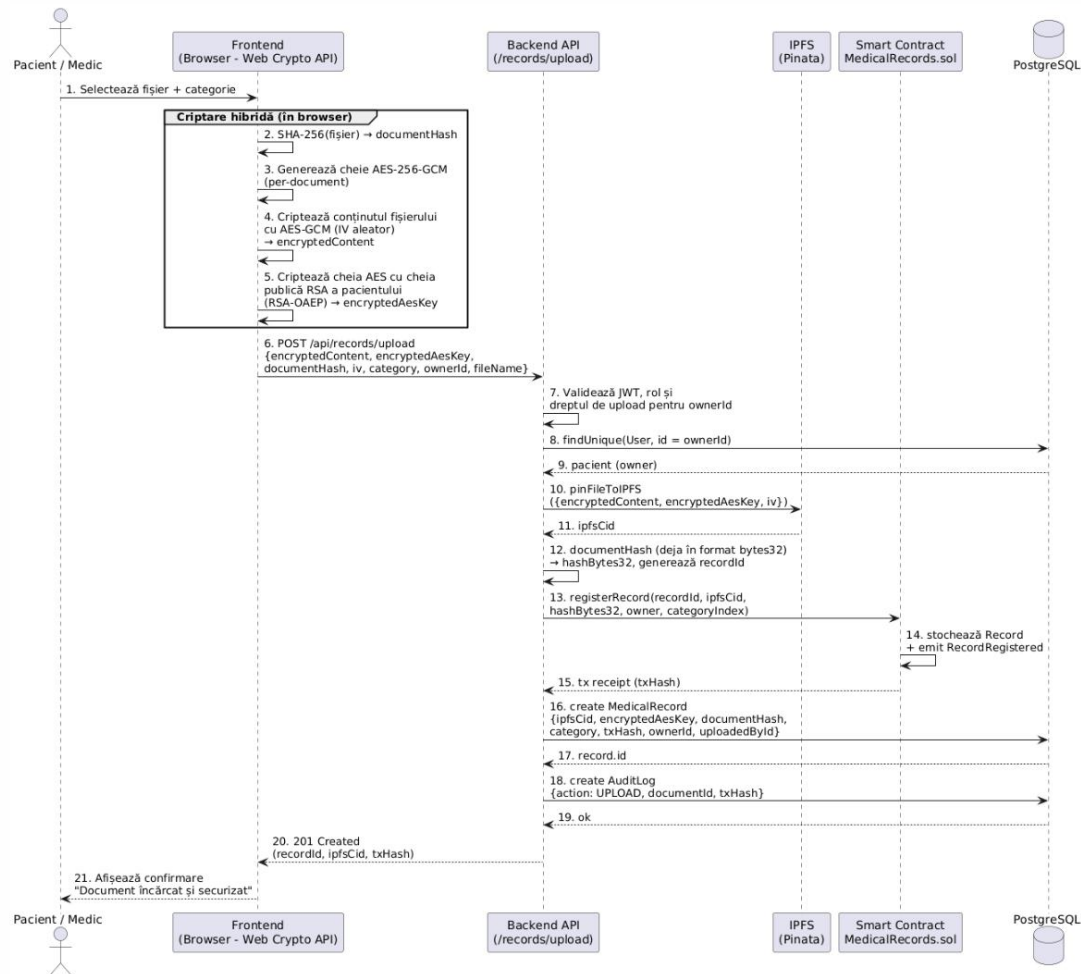


Figura 3.3.2.1: Diagrama de secvență pentru încărcarea documentelor

Interfața dedicată încărcării documentelor utilizează un formular prin care utilizatorul selectează fișierul pe care dorește să îl încarce, categoria din care acesta face parte și, în cazul medicilor, CNP-ul pacientului destinat (vezi figurile 3.3.2.2 și 3.3.2.3). La începutul procesului de încărcare, fișierul este preluat sub formă de `ArrayBuffer` și se calculează hash-ul SHA-256 asupra conținutului, generând astfel amprenta necesară verificării integrității. În etapa următoare se generează o cheie simetrică AES-256 și un vector de 96 de biți pentru criptarea conținutului și pentru a garanta că pacientul corespunzător documentului încărcat este cel care deține controlul asupra decriptării ulterioare. Această funcționalitate a fost implementată prin criptarea cheii AES generate cu cheia RSA-OAEP a pacientului.

Figura 3.3.2.2: Formularul de încărcare pentru pacienți

Figura 3.3.2.3: Formularul de încărcare al medicilor

În urma acestor operațiuni, server-ul primește datele și integrează conținutul criptat, cheia AES și vectorul într-un obiect JSON, pe care îl încarcă pe IPFS prin intermediul serviciului Pinata. Backend-ul obține apoi CID-ul aferent conținutului încărcat și inițializează o tranzacție pe blockchain folosind contractul MedicalRecords. După ce tranzacția este confirmată on-chain, metadatele documentului și hash-ul tranzacției sunt salvate în baza de date PostgreSQL.

3.3.3 Descărcarea și verificarea integrității

Afișarea documentelor este realizată printr-o interfață vizuală de tip tabel paginat, ce permite căutarea după denumire și filtrarea în funcție de categoria din care face parte documentul (vezi figura 3.3.3). La accesarea unei înregistrări individuale, backend-ul validează drepturile utilizatorului și returnează CID-ul și cheia AES în formă criptată, declanșând mecanismul de audit.

Procesul de restaurare a documentului începe prin utilizarea cheii private RSA pentru a decripta cheia simetrică, urmată apoi de etapa decriptării propriu-zise a fișierului. Fișierele care trec testul de integritate bazat pe recalcularea hash-ului SHA-256 sunt procesate în funcție de tip: documentele de tip PDF, JPEG sau PNG sunt deschise într-un tab nou pentru vizualizare directă, în timp ce restul formatelor declanșează descărcarea locală pe dispozitivul utilizatorului. Aceste funcționalități sunt disponibile doar pentru pacienți deoarece medicii nu au acces la cheia privată RSA a pacientului, necesară pentru decriptarea informației.

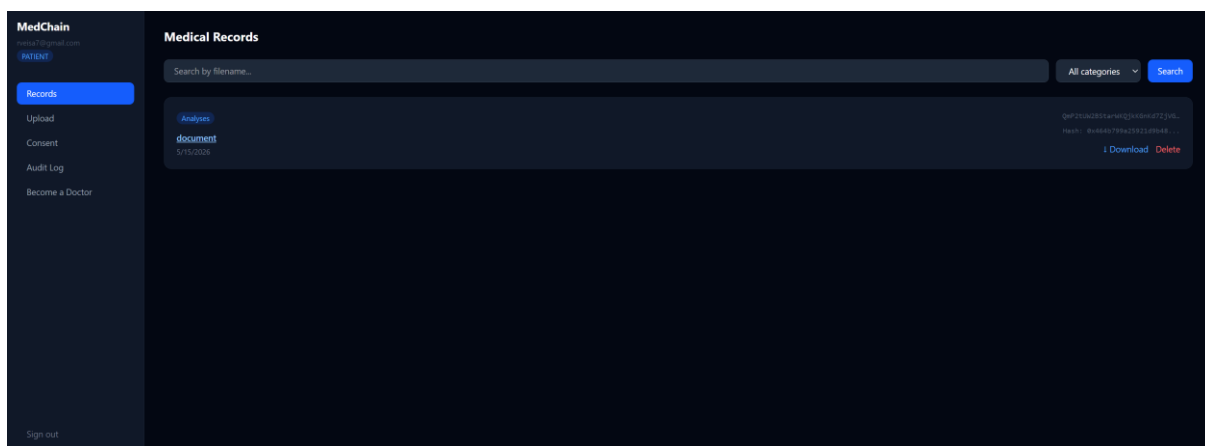


Figura 3.3.3: Interfața pentru documente

3.3.4 Consimțământul și auditul accesului

Pentru a permite pacienților să aibă un control mai bun asupra datelor proprii, am implementat un mecanism de consimțământ pe patru categorii (ANALYSES, PRESCRIPTIONS, IMAGING, CONSULTATIONS) și o interfață de tip tablou de bord pentru managementul permisiunilor.

Din perspectiva pacientului, procesul începe prin introducerea CNP-ului medicului căruia dorește să îi ofere acces. Odată confirmată existența acestuia și starea contului, server-ul interacționează cu rețeaua blockchain prin intermediul bibliotecii ethers.js, apelând funcția corespunzătoare din contractul Consent. Parametrii transmiși sunt apoi înregistrați on-chain și, odată ce tranzacția este confirmată, starea este actualizată sincron off-chain. Pașii exacți ai acestui proces, de la inițierea cererii din interfața utilizatorului până la sincronizarea datelor între baza de date relațională (PostgreSQL) și smart contract, sunt detaliați în diagrama de secvență din figura 3.3.4.1.

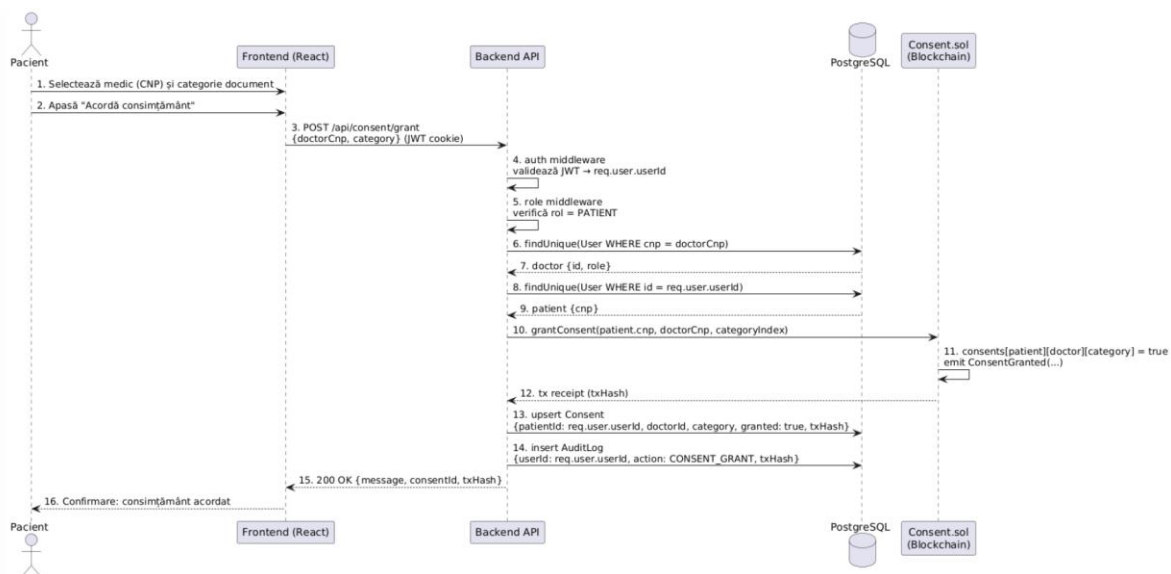


Figura 3.3.4.1: Diagrama de secvență pentru acordarea consimțământului

În paralel cu gestionarea permisiunilor, sistemul realizează și un audit pentru activități pentru a asigura o transparență totală, acesta fiind vizibil în pagina de audit a platformei (vezi figura 3.3.4.2). Interacțiunile precum acordarea consimțământului unui medic sau accesarea propriu-zisă a unui fișier generează atât o înregistrare în tabela AuditLog cât și o înregistrare on-chain prin smart contract-ul corespunzător.

MedChain Patient Records Upload Consent Audit Log Become a Doctor	Audit Log		
	VIEW	Radu Alexandrescu 831227F1-45C8-4272-8F28-746328148941	5/18/2024, 6:57:08 PM
	VIEW	Radu Alexandrescu 831227F1-45C8-4272-8F28-746328148941	5/18/2024, 6:57:04 PM
	VIEW	Radu Alexandrescu 831227F1-45C8-4272-8F28-746328148941	5/18/2024, 6:57:02 PM
	VIEW	Radu Alexandrescu 831227F1-45C8-4272-8F28-746328148941	5/18/2024, 6:57:00 PM
	VIEW	Radu Alexandrescu 831227F1-45C8-4272-8F28-746328148941	5/18/2024, 6:56:51 PM
	UPLOAD	Radu Alexandrescu 831227F1-45C8-4272-8F28-746328148941	5/18/2024, 5:36:58 PM
	UPLOAD	Radu Alexandrescu 261396C2-5963-4663-8F77-494712616478	5/18/2024, 5:27:17 PM
	REVOKE	Radu Alexandrescu	5/18/2024, 5:25:38 PM

Figura 3.3.4.2: Pagina de audit

3.3.5 Verificarea medicilor și panoul de admin

Managementul platformei este realizat printr-un panou de control dedicat, accesibil exclusiv utilizatorilor care au rolul de admin (vezi figura 3.3.5.1). La momentul autentificării, sistemul detectează automat rolul administratorului și îl redirecționează către ruta protejată

/admin. Interfața este lipsită de funcționalități cu privire la gestionarea documentelor sau a consimțământului pentru a asigura o separare între datele clinice și operațiunile administrative.

Sistemul de administrare este alcătuit din patru module, primul fiind reprezentat de un tablou de bord pentru monitorizarea ecosistemului. Acesta afișează statistici realizate în timp real precum: numărul total de utilizatori, numărul fișierelor stocate, consimțămintele active și volumul înregistrărilor din audit. Cel de-al doilea modul este cel utilizat în managementul global al conturilor, oferind o listă completă a utilizatorilor cu datele lor de identificare, rolul deținut și data înregistrării. În această secțiune, administratorul deține puterea de a șterge conturile utilizatorilor.

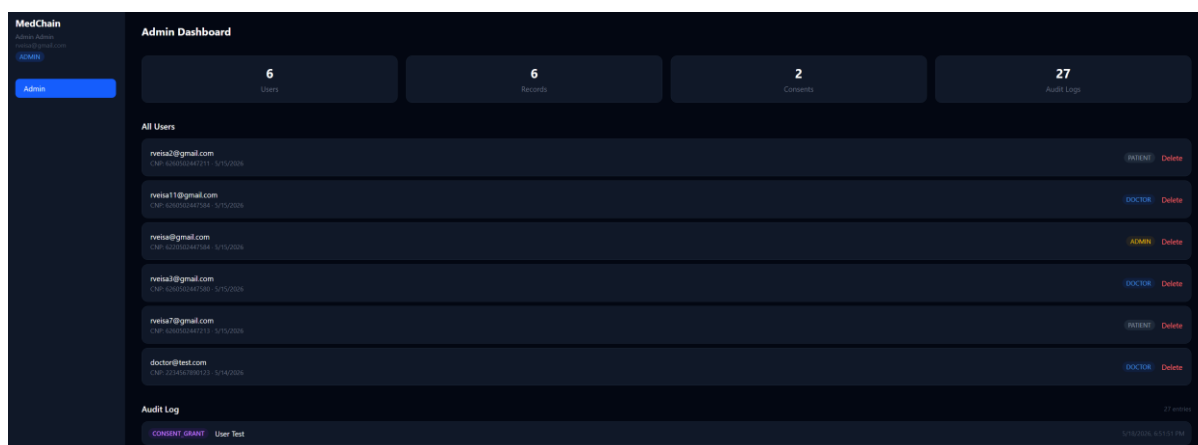


Figura 3.3.5.1: *Pagina administratorilor*

Cea mai importantă componentă a paginii o reprezintă modulul de gestionare a cererilor de acordare a rolului de medic. Platforma a fost proiectată astfel încât atunci când un utilizator își creează contul, acesta primește automat rol de pacient în sistem, indiferent dacă acesta este medic sau pacient. Pentru ca un utilizator să obțină privilegii de medic, acesta trebuie să utilizeze pagina specifică acestei situații pe platformă (vezi figura 3.3.5.2) și să trimită o cerere către sistem cu următoarele date: numele complet, specializarea și codul de parafă. Un administrator poate vedea aceste cereri în interfața proprie și poate accepta sau respinge solicitarea. În cazul în care cererea este aprobată, sistemul actualizează simultan statusul cererii în baza de date și rolul utilizatorului în sistem.

The image shows a web application interface for MedChain. On the left is a dark sidebar with the MedChain logo and a list of menu items: PATIENT, Records, Upload, Consent, Audit Log, and Become a Doctor (which is highlighted in blue). At the bottom of the sidebar is a 'Sign out' link. The main content area has a title 'Become a Doctor' and a form with three input fields: 'Full name' (containing 'Dr. Ion Popescu'), 'Seal code' (containing 'AB1234'), and 'Specialization' (containing 'Cardiology'). Below these fields is a blue 'Submit Request' button.

Figura 3.3.5.2: *Formularul de identificare pentru medici*

Ultimul modul al paginii administratorilor este cel destinat vizualizării intrărilor de audit. Spre deosebire de log-urile vizibile pacienților și medicilor, care sunt filtrate pe baza identității proprii, intrările pe care le poate accesa administratorul cuprind toate acțiunile realizate de către toți utilizatorii. Acestea sunt prezentate în pagini de câte 20 de intrări fiecare, conținând numele complet al utilizatorului, acțiunea și data la care a avut loc evenimentul.

Capitolul 4 - Analiza securității și a fezabilității

4.1 Analiza amenințărilor

Evaluarea securității sistemului a fost realizată folosind metodologia STRIDE (Microsoft, 2022), identificând riscurile specifice fiecărei componente și prezentând contramăsurile implementate.

Amenințare	Scenariu concret	Contramăsuri
Spoofing	Furt de sesiune prin atac XSS	Token JWT stocat în cookie HttpOnly și Secure, inaccesibil din JavaScript
Tampering	Modificarea unui document medical	Criptare AES-256-GCM și verificare integritate prin hash SHA-256 la fiecare descărcare
Repudiation	Un pacient neagă accesarea unui document	Audit dublu: PostgreSQL pentru acces rapid și smart contract AuditLog pentru trasabilitate on-chain
Information Disclosure	Acces neautorizat la fișierele de pe IPFS	Criptare end-to-end (nu poate fi accesat conținutul în clar)
Denial of Service	Supraîncărcarea server-ului prin încărcarea de fișiere foarte mari	Limitarea dimensiunii payload-ului la 50 MB și validarea upload-urilor
Elevation of Privilege	Un pacient încearcă să acceseze documentele altui pacient	Verificarea ownership-ului pe fiecare endpoint și validarea existenței consimțământului activ pentru medici

Tabelul 4.1: *Analiza STRIDE*

4.2 Confidențialitatea datelor și conformitatea GDPR

Integrarea tehnologiilor de blockchain într-un ecosistem medical a necesitat menținerea unui echilibru atent între caracterul imuabil al rețelei și cerințele GDPR.

Deoarece ștergerea fizică a datelor de pe blockchain este imposibilă, am ales să implementez un mecanism care, la solicitarea ștergerii unui document, elimină din baza de date intrarea asociată, inclusiv cheia AES. Fără acest element, conținutul de pe IPFS devine irecuperabil, deși referința CID rămâne stocată on-chain. Această decizie de proiectare se conformează cu dreptul individului de a cere ștergerea datelor cu caracter personal.

4.3 Analiza costurilor de tranzacție

Pentru evaluarea fezabilității economice a soluției, costurile de tranzacție ale contractelor au fost măsurate folosind hardhat-gas-reporter în cadrul suitei de teste automate. Rezultatele prezentate în tabelul de mai jos reflectă consumul real de gas pentru funcțiile importante ale platformei:

Operațiune	Contract	Gas consumat (avg)	% din limita blocului
Deploy	MedicalRecords	806.264	2,7%
Deploy	Consent	382.365	1,3%
Deploy	AuditLog	275.695	0,9%
Înregistrare document	MedicalRecords	141.103	-
Acordare consimțământ	Consent	50.188	-
Revocare consimțământ	Consent	28.310	-
Logare acces	AuditLog	29.418	-

Tabelul 4.3: *Rezultate analiza costurilor*

Valorile obținute indică un consum modest de gas pentru operațiunile frecvente ale platformei, comparativ cu costul de referință al unei tranzacții simple pe rețeaua Ethereum, care implică un cost de 21.000 de unități de gas (Ethereum Foundation, 2026). Logarea accesului (29.418 gas) reprezintă operațiunea on-chain cea mai frecventă, fiind declanșată la fiecare vizualizare a unui document și situându-se imediat deasupra pragului minim, iar operațiunile de gestionare a consimțământului (acordare - 50.188 gas, revocare - 28.310 gas) au de asemenea o frecvență ridicată în utilizarea curentă a platformei. Costul mai mare al acordării este determinat de inițializarea unei noi poziții în mapping-ul de stare al contractului Consent (scriere dintr-o valoare implicită zero), spre deosebire de revocare, care actualizează o poziție deja existentă. Înregistrarea documentelor noi (141.103 gas) implică un consum mai ridicat, justificat de numărul mai mare de câmpuri păstrate on-chain prin operații de scriere în storage-ul contractului. În ceea ce privește deployment-ul contractelor, aceasta este o operațiune unică, realizată o singură dată la lansarea platformei, și nu afectează costurile operaționale curente. Astfel, toate operațiunile se încadrează sub 3% din limita unui bloc, fapt ce confirmă viabilitatea economică a soluției pentru un scenariu de utilizare reală.

Capitolul 5 - Concluzii

Lucrarea a demonstrat fezabilitatea platformei ca soluție de gestionare securizată a datelor medicale prin combinarea criptografiei aplicate în browser cu proprietățile de imuabilitate și transparență ale tehnologiei blockchain. Contribuția principală rămâne arhitectura care redă pacienților controlul deplin asupra istoricului clinic.

Implementarea actuală prezintă câteva limitări cunoscute. Utilizarea unei rețele locale Hardhat este potrivită pentru faza de prototip, datele nebeneficiind de descentralizarea reală a unei rețele publice. De asemenea, prezența CNP-ului în contractele inteligente constituie un risc de confidențialitate pe termen lung, ce necesită o soluție de anonimizare. Astfel, cele mai relevante direcții viitoare de dezvoltare sunt migrarea pe o rețea Layer 2 precum Polygon pentru producție, extinderea mecanismului de vizualizare a documentelor pentru medici și înlocuirea identificatorului personal stocat on-chain cu un altul artificial.

Bibliografie

- Barker, E., & Roginsky, A. (2019). *Transitioning the Use of Cryptographic Algorithms and Key Lengths*. NIST Special Publication. Preluat de pe <https://doi.org/10.6028/NIST.SP.800-131Ar2>
- Epic Systems Corporation. (2026, March 23). *Security & Privacy Policies*. Preluat pe May 15, 2026, de pe Epic: <https://www.epic.com/en-us/privacypolicies/>
- Ethereum Foundation. (2026, February 16). *Gas and fees*. Preluat pe June 10, 2026, de pe Ethereum: <https://ethereum.org/developers/docs/gas/>
- Ethereum Foundation. (2026, February 25). *Introduction to Smart Contracts*. Preluat pe May 16, 2026, de pe Ethereum: <https://ethereum.org/developers/docs/smart-contracts/>
- Jonsson, J., & Kaliski, B. (2003). *Public-Key Cryptography Standards (PKCS) #1: RSA Cryptography Specifications Version 2.1*. Request for Comments. Preluat de pe <https://www.rfc-editor.org/rfc/rfc3447>
- Microsoft. (2022, August 25). *Microsoft Threat Modeling Tool threats*. Preluat pe May 16, 2026, de pe Microsoft Learn: <https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats>
- Moriarty, K. M., Kaliski, B., & Rusch, A. (2017). *Password-Based Cryptography Specification Version 2.1*. Request for Comments. Preluat de pe <https://www.rfc-editor.org/rfc/rfc8018>
- Nakamoto, S. (2008). Bitcoin: A Peer-to-Peer Electronic Cash System. Preluat de pe <https://bitcoin.org/bitcoin.pdf>
- National Institute of Standards and Technology. (2001). *Advanced Encryption Standard (AES)*. Federal Information Processing Standard. Preluat de pe <https://doi.org/10.6028/NIST.FIPS.197-upd1>
- National Institute of Standards and Technology. (2015). *Secure Hash Standard (SHS)*. Federal Information Processing Standard. Preluat de pe <https://doi.org/10.6028/NIST.FIPS.180-4>
- Oracle Health. (n.d.). *Patient Portal Solution Brief*. Preluat pe May 15, 2026, de pe Oracle: <https://www.oracle.com/a/ocom/docs/industries/healthcare/patient-portal-solution-brief.pdf>
- Protocol Labs. (n.d.). *What is IPFS?* Preluat pe May 16, 2026, de pe IPFS: <https://docs.ipfs.tech/concepts/what-is-ipfs/>